



BÀI 02: NGÔN NGỮ LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG JAVA

- ❑ Giới thiệu về Java
- ❑ Các dạng chương trình ứng dụng java
- ❑ Cấu trúc chung của một chương trình java
- ❑ Biên dịch và thông dịch chương trình java
- ❑ Các phần tử cơ bản của Java



Giới thiệu về Java

- ❑ Sự phát triển của Java
 - Được phát triển bởi Sun Microsystems 6/1991 với tên gọi là Oak (Các tác giả đầu tiên gồm: James Gosling, Mike Sheridan, và Patrick Naughton)
 - Mục tiêu ban đầu của dự án là tạo ra một ngôn ngữ chạy trên các thiết bị điện tử như tivi hay các đầu thu phát (set-top boxes).
 - Tên đầu tiên họ đặt cho ngôn ngữ này là “Greentalk”
 - Năm 1995 được đổi tên là Java
- ❑ Tương tự ngôn ngữ C++ nhưng uyển chuyển hơn
- ❑ Hướng đến lập trình Web



Giới thiệu về Java

Phiên bản	Kiểu	Ngày phát hành	Kết thúc cập nhật công khai (miễn phí)
JDK 1.0		23 tháng 1 năm 1996	tháng 5 năm 1996
JDK 1.1		18 tháng 2 năm 1997	Tháng 10 năm 2002
J2SE 1.2		ngày 4 tháng 12 năm 1998	Tháng 11 năm 2003
J2SE 1.3		ngày 8 tháng 5 năm 2000	tháng 3 năm 2006
J2SE 1.4		13 tháng 2 năm 2002	Tháng 10 năm 2008
J2SE 5.0		30 tháng 9 năm 2004	Tháng 10 năm 2009
Java SE 6		Ngày 11 tháng 12 năm 2006	tháng 4 năm 2013
Java SE 7		Ngày 28 tháng 7 năm 2011	tháng 7 năm 2015
Java SE 8	LTS	Ngày 18 tháng 3 năm 2014	Tháng 4 năm 2019 đối với Oracle
Java SE 9		Ngày 21 tháng 9 năm 2017	tháng 3 năm 2018
Java SE 10		Ngày 20 tháng 3 năm 2018	tháng 9 năm 2018
Java SE 11	LTS	Ngày 25 tháng 9 năm 2018	Tháng 4 năm 2019 cho Oracle Tháng 10 năm 2024 cho Red Hat
Java SE 12		Ngày 19 tháng 3 năm 2019	tháng 9 năm 2019



Giới thiệu về Java

Phiên bản	Kiểu	Ngày phát hành	Kết thúc cập nhật công khai (miễn phí)
Java SE 13		Ngày 17 tháng 9 năm 2019	tháng 3 năm 2020
Java SE 14		Ngày 17 tháng 3 năm 2020	tháng 9 năm 2020
Java SE 15		Ngày 16 tháng 9 năm 2020	tháng 3 năm 2021
Java SE 16		Ngày 16 tháng 3 năm 2021	tháng 9 năm 2021
Java SE 17	LTS	Ngày 14 tháng 9 năm 2021	Tháng 9 năm 2024 cho Oracle
Java SE 18		Ngày 22 tháng 3 năm 2022	tháng 9 năm 2022
Java SE 19		Ngày 20 tháng 9 năm 2022	tháng 3 năm 2023
Java SE 20		Ngày 21 tháng 3 năm 2023	tháng 9 năm 2023
Java SE 21	LTS	Ngày 19 tháng 9 năm 2023	Tháng 9 năm 2026 cho Oracle Tháng 12 năm 2029 cho Red Hat
Java SE 22		Ngày 19 tháng 3 năm 2024	tháng 9 năm 2024
Java SE 23		tháng 9 năm 2024	tháng 3 năm 2025
Java SE 24		tháng 3 năm 2025	tháng 9 năm 2025



Giới thiệu về Java

❑ Một số đặc trưng chính của Java:

■ Đơn giản

- ✓ *Không sử dụng con trỏ (tường minh)*
- ✓ *Không chuyển kiểu tự động*
- ✓ *Không định nghĩa lại phép toán (chồng toán tử)*
- ✓ *Không thừa kế bội,...*

■ Hướng đối tượng

- ✓ *Là ngôn ngữ lập trình hướng đối tượng thuần khiết*
- ✓ *Phát triển lớp trên các lớp sẵn có*



Giới thiệu về Java

❑ Một số đặc trưng chính của Java:

■ Đơn giản

- ✓ *Không sử dụng con trỏ (tường minh)*
- ✓ *Không chuyển kiểu tự động*
- ✓ *Không định nghĩa lại phép toán (chồng toán tử)*
- ✓ *Không thừa kế bội,...*

■ Hướng đối tượng

- ✓ *Là ngôn ngữ lập trình hướng đối tượng thuần khiết*
- ✓ *Phát triển lớp trên các lớp sẵn có*



Giới thiệu về Java

- ❑ Một số đặc trưng chính của Java:
 - Độc lập nền (Platform-independent): Hệ thư viện
 - Độc lập với kiến trúc máy tính
 - Độc lập với hệ điều hành
 - Ngôn ngữ mạnh, hiệu năng cao
 - Bảo mật
 - Java đưa ra nhiều giải pháp hỗ trợ an toàn, bảo mật



Các dạng chương trình ứng dụng java

- ☐ Các chương trình ứng dụng độc lập (console), **StandAloneApp.java**

- ☐ Các chương trình ứng dụng nhúng (applet), **AppletApp.java**

- ☐ Kết hợp cả 2 loại trên (Ứng dụng hỗn hợp), **MixedApp.java**



Cấu trúc chung của một chương trình java

// Filename: NewApp.java

```
// Phần 1: Tùy chọn  
// Định nghĩa gói  
package GoiNhaTruong;
```

```
// Phần 2: (0 hoặc nhiều hơn)  
// các gói cần sử dụng  
import java.io.*;
```

```
// Phần 3: (0 hoặc nhiều hơn)  
// Định nghĩa các lớp và các interface  
public class NewApp{ ...}  
class C1{ ...}  
interface I1{ ...}  
//  
class Cn { ...}  
interface Im{ ...}
```

Định nghĩa gói thông qua định danh của gói (*package*).
Tất cả các lớp, các *interface* được định nghĩa trong tệp chứa gói này đều thuộc gói đó.
Mặc định (không định nghĩa) thì các định nghĩa ở tệp này sẽ thuộc vào gói mặc định.

Một số lệnh nhập import.
Phần này có thể không có, hoặc có nhiều lệnh nhập

Một số định nghĩa lớp và *interface*
có thể định nghĩa theo thứ tự bất kỳ



Cấu trúc chung của một chương trình java

```
1 // Tên file : Hello.java
2 /* Tác giả : Barak Obama */
3
4 public class Hello
5 {
6     // Phương thức main, điểm bắt đầu của chương trình
7     public static void main( String args[] )
8     {
9         System.out.println( "Hello World" );
10    } // Kết thúc phương thức main
11 } // Khai báo lớp
12
13
```

Tên lớp chứa hàm main phải giống tên file

Điểm bắt đầu và kết thúc của lớp

Dấu hiệu chú thích =>
Làm cho chương trình dễ hiểu hơn. Trình biên dịch sẽ bỏ qua những dòng có dấu chú thích

Khai báo lớp
Mỗi CT phải có ít nhất một khai báo lớp

Hiện thị dãy ký tự ra màn hình

Phương thức main() sẽ được gọi đầu tiên. Mỗi CT thực thi phải có một phương thức main()

Các câu lệnh phải kết thúc bằng dấu chấm phẩy



Cấu trúc chung của một chương trình java

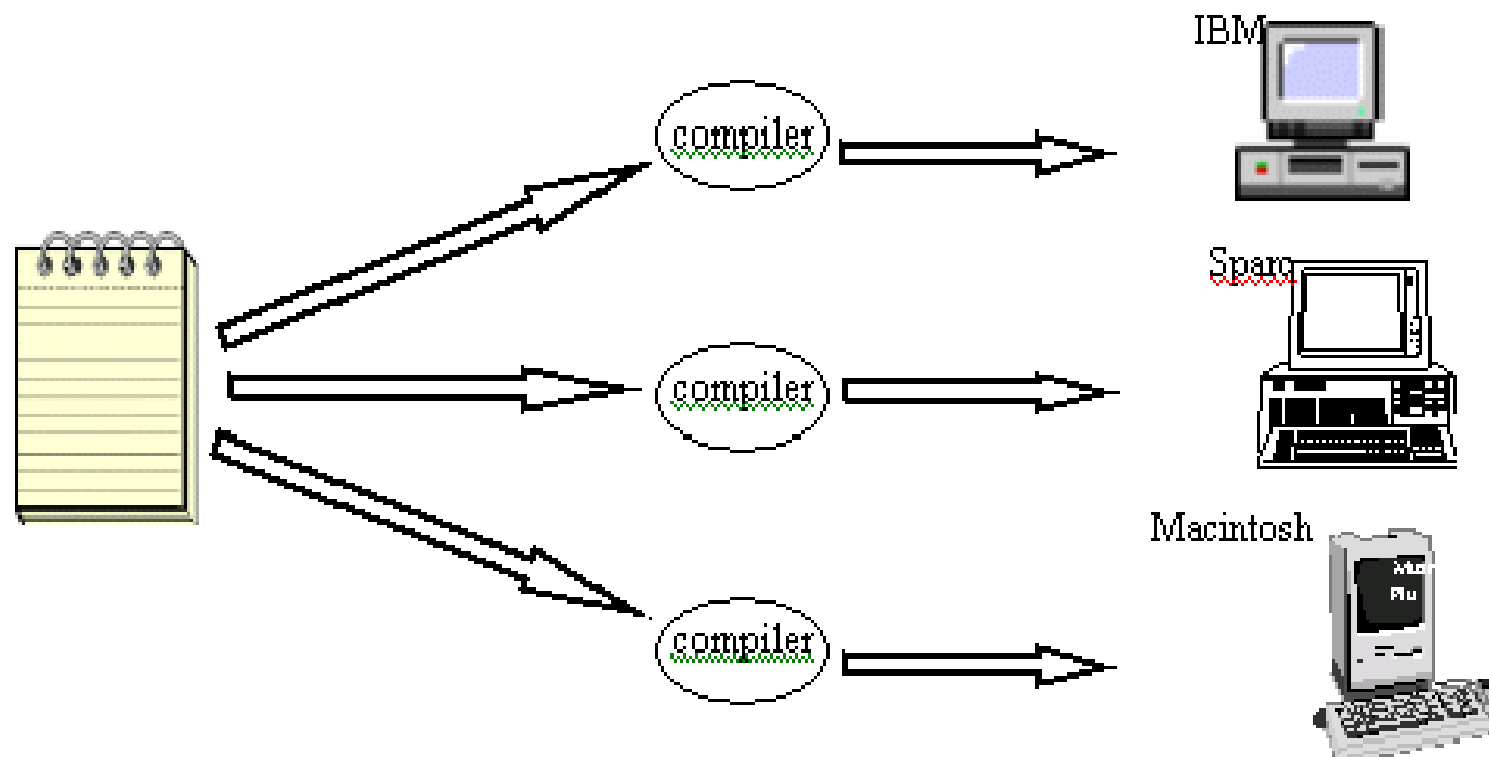
- ❑ **Phương thức main():** là điểm bắt đầu thực thi một ứng dụng.

- ❑ Mỗi ứng dụng Java phải chứa một phương thức main có dạng như sau: **public static void main(String[] args)**

- ❑ Phương thức main chứa ba bổ từ đặc tả sau:
 - **public:** chỉ ra rằng phương thức main có thể được gọi bởi bất kỳ đối tượng nào.
 - **static:** chỉ ra rằng phương thức main là một phương thức lớp.
 - **void:** chỉ ra rằng phương thức main sẽ không trả về bất kỳ một giá trị nào.

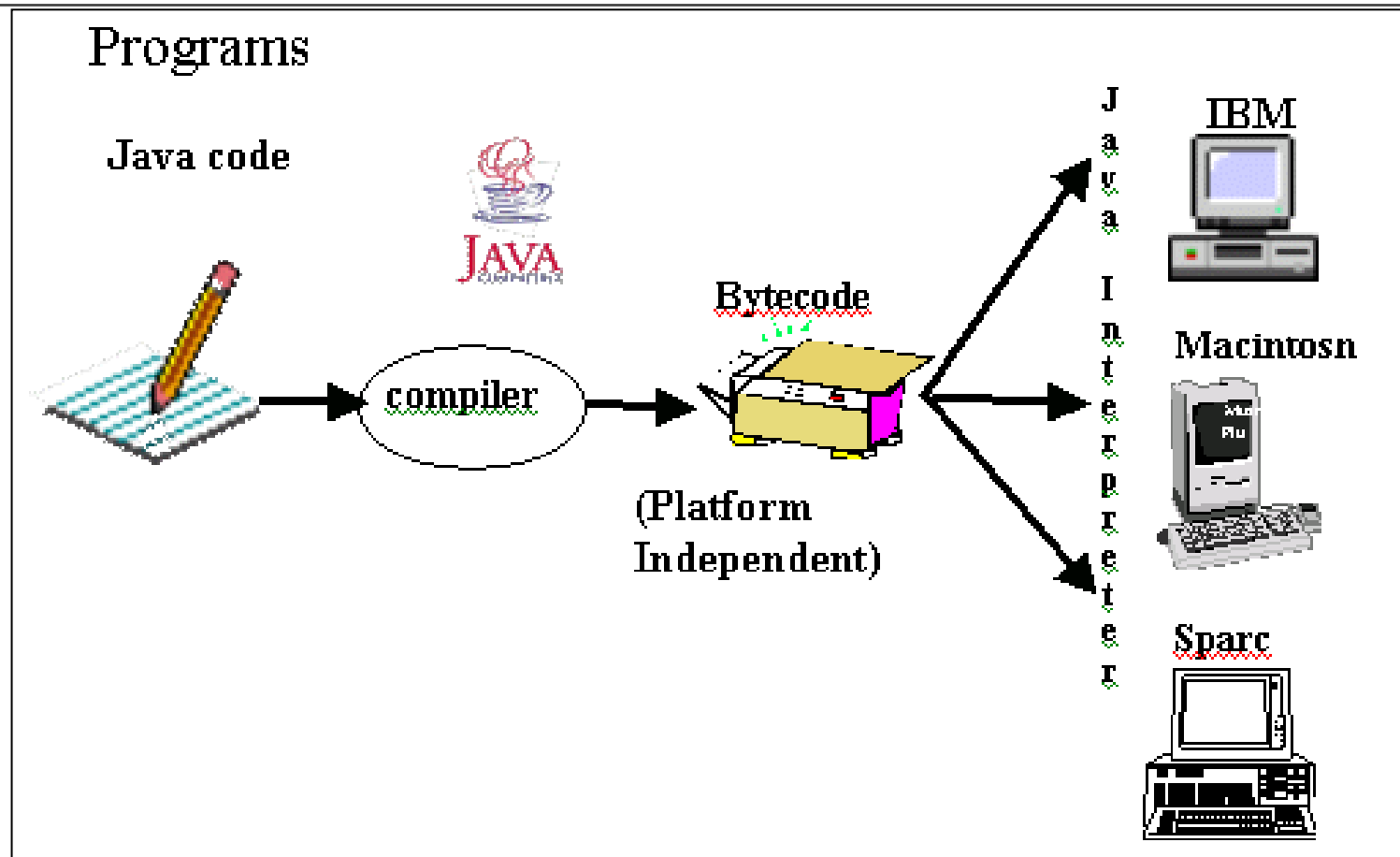


Biên dịch và thông dịch chương trình java



Biên dịch chương trình theo phương pháp truyền thống

Biên dịch và thông dịch chương trình java



Biên dịch trong Java



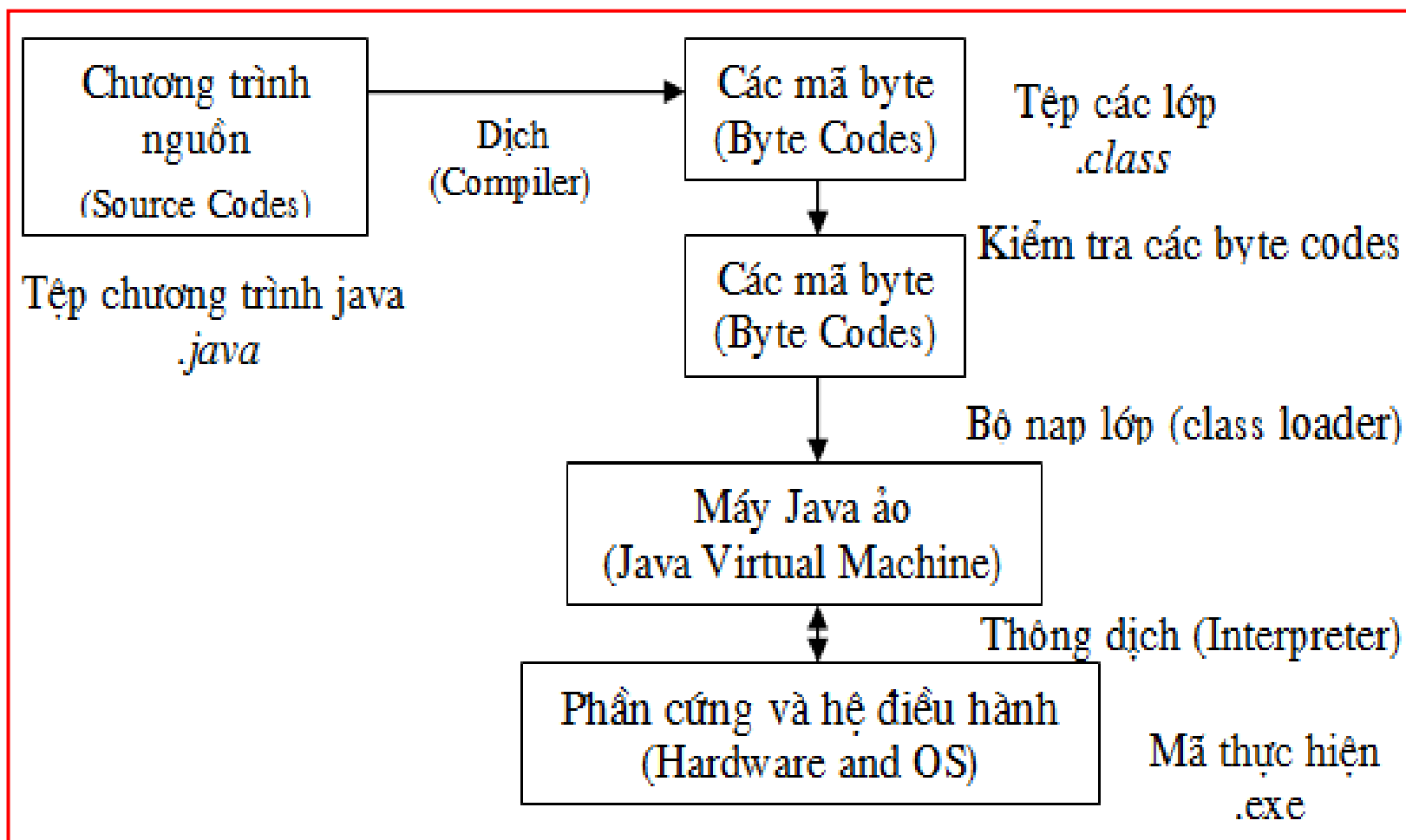
Cấu trúc chung của một chương trình java

Các bước để dịch một chương trình Java:

- Trình biên dịch chuyển mã nguồn thành tập các lệnh không phụ thuộc vào phần cứng cụ thể
- Trình thông dịch trên mỗi máy chuyển tập lệnh này thành chương trình thực thi
- Máy ảo tạo ra một môi trường để thực thi các lệnh bằng cách:
 - Nạp các file .class
 - Quản lý bộ nhớ
 - Dọn “rác”



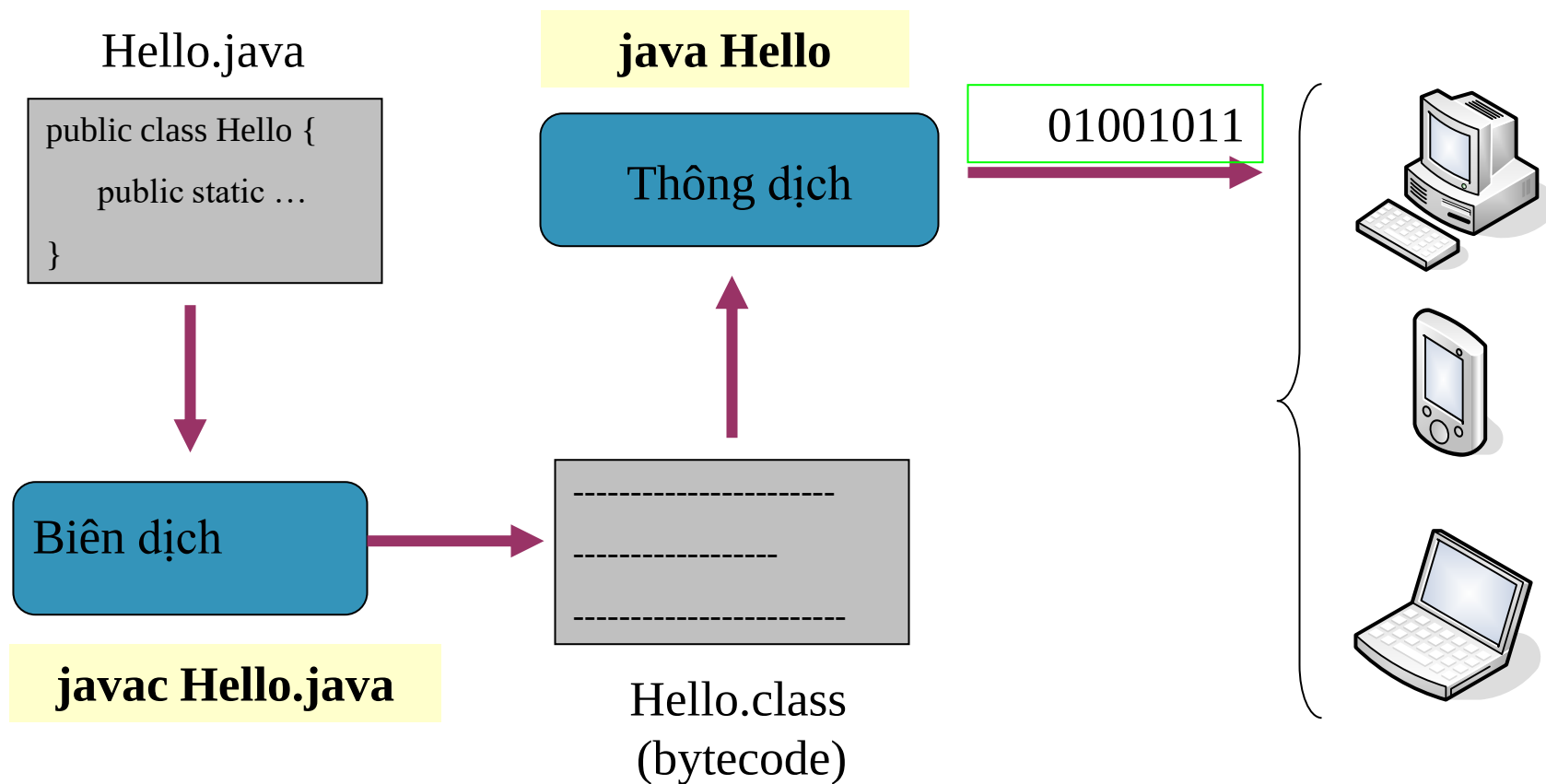
Biên dịch và thông dịch chương trình java



Quá trình dịch và thông dịch chương trình Java



Biên dịch và thông dịch chương trình java



Các bước phát triển một chương trình bằng Java



Các thành phần cơ sở của Java

- ❑ Các thành phần cơ sở của Java
- ❑ Kiểu dữ liệu nguyên thủy
- ❑ Cấu trúc dữ liệu mảng
- ❑ Các câu lệnh cơ bản
- ❑ Các câu lệnh điều khiển



Các thành phần cơ sở của Java

❑ Từ khoá:

abstract	boolean	break	byte
case	catch	char	class
const	continue	default	do
double	else	extends	final
finally	float	for	goto
if	implements	import	instanceof
int	interface	long	native
new	package	private	protected
public	return	short	static
super	switch	synchronized	this
throw	throws	transient	try
void	volatile	while	



Các thành phần cơ sở của Java

Từ khóa cho các kiểu dữ liệu cơ bản : **byte, short, int, long, float, double, char, boolean.**

Từ khóa cho phát biểu lặp: **do, while, for, break, continue.**

Từ khóa cho phát biểu rẽ nhánh: **if, else, switch, case, default, break.**

Từ khóa đặc tả đặc tính một **method**: **private, public, protected, final, static, abstract, synchronized.**

Từ khóa liên quan đến phương thức: **return, void.**

Từ khóa liên quan đến gói: **package, import.**

Từ khóa cho việc quản lý lỗi: **try, catch, finally, throw, throws.**

Từ khóa liên quan đến đối tượng: **new, extends, implements, class, instanceof, this, super.**



Các thành phần cơ sở của Java

❑ Định danh:

- Định danh là dùng biểu diễn tên của biến, của phương thức, của lớp.
- Trong Java, định danh có thể sử dụng ký tự chữ, ký tự số và ký tự dấu.
- Ký tự đầu tiên phải là ký tự chữ, dấu gạch dưới (_), hoặc dấu dollar (\$).
- Có sự phân biệt giữa ký tự chữ hoa và chữ thường.
- Định danh không được trùng với các từ khóa.

Ví dụ: Hello, _prime, var8, tvLang



Các thành phần cơ sở của Java

❑ Biến:

- Biến là vùng nhớ dùng để lưu trữ các giá trị của chương trình.
- Mỗi biến gắn liền với một kiểu dữ liệu và một định danh duy nhất gọi là tên biến.
- Trong java, biến có thể được khai báo ở bất kỳ nơi đâu trong chương trình.
- Cách khai báo:
 <kiểu_dữ_liệu> <tên_biến>;
 <kiểu_dữ_liệu> <tên_biến> = <giá_trị>;
- Gán giá trị cho biến:
 <tên_biến> = <giá_trị>;



Các thành phần cơ sở của Java

❑ Hằng:

- ❑ Hằng là một giá trị bất biến trong chương trình
- ❑ Tên hằng được đặt theo quy ước giống như tên biến
- ❑ Có thể có hậu tố: l, L, f, F, d, D
 - ❑ l hoặc L: chỉ số nguyên dài (long).
 - ❑ f hoặc F: chỉ số thực dấu phẩy động (float).
 - ❑ d hoặc D: chỉ số thực gấp đôi hoặc dấu phẩy động gấp đôi (double).
- ❑ Ví dụ:
int i=1;
long i=1L;
- ❑ Hằng ký tự: là một ký tự đơn nằm giữa 2 dấu nháy đơn.



Các thành phần cơ sở của Java

□ Hằng:

- Hằng chuỗi: là tập hợp các ký tự được đặt giữa hai dấu nháy kép “”. Một hằng chuỗi không có ký tự nào là một hằng chuỗi rỗng.
- Ví dụ: “Hello Wolrd”
- Lưu ý: Hằng chuỗi không phải là một kiểu dữ liệu cơ sở nhưng vẫn được khai báo và sử dụng trong các chương trình



Toán tử (operator)

❑ Toán tử số học:

Toán tử	Ý nghĩa
+	Cộng
-	Trừ
*	Nhân
/	Chia
%	Chia dư
++	Tăng 1
--	Giảm 1



Toán tử (operator)

❑ Toán tử quan hệ & logic:

Toán tử	Ý nghĩa
==	So sánh bằng
!=	So sánh khác
>	So sánh lớn hơn
<	So sánh nhỏ hơn
>=	So sánh lớn hơn hay bằng
<=	So sánh nhỏ hơn hay bằng
	OR (biểu thức logic)
&&	AND (biểu thức logic)
!	NOT (biểu thức logic)



Toán tử (operator)

❑ Toán tử gán (assignment):

Toán tử	Ví dụ	Giải thích
=	int c=3, d=5, c=4;	
+=	c+=7	c=c+7
-=	d-=4	d=d-4
=	e=5	e=e*5
/=	f/=3	f=f/3



Toán tử (operator)

❑ Toán tử điều kiện:

<điều kiện> ? <biểu thức 1> : <biểu thức 2>

int x = 10;

int y = 20;

int Z = (x < y) ? 30 : 40;



Toán tử (operator)

□ Độ ưu tiên của các phép toán:

Độ ưu tiên	Tên gọi	Các phép toán	Quy tắc kết hợp thực hiện
1	Phép toán 1 ngôi hậu tố (postfix)	[] . (tham_so) exp++ exp--	Từ trái qua phải
2	Phép toán 1 ngôi tiền tố (prefix)	++exp --exp +exp -exp ~ !	Từ phải qua trái
3	Tạo lập đối tượng, ép kiểu	new() (type)	Từ phải qua trái
4	Loại phép nhân	* / %	Từ trái qua phải
5	Loại phép cộng	+ -	Từ trái qua phải
6	Chuyển dịch	<< >> >>>	Từ trái qua phải
7	Phép toán quan hệ	< <= > >= instanceof	Từ trái qua phải
8	Phép so sánh đẳng thức	== !=	Từ trái qua phải
9	Phép AND	&	Từ trái qua phải
10	Phép XOR	^	Từ trái qua phải
11	Phép OR		Từ trái qua phải
12	Phép AND	&&	Từ trái qua phải
13	Phép hoặc (OR) logic		Từ trái qua phải
14	Phép toán điều kiện	?:	Từ trái qua phải
15	Các phép gán	= += -= *= /= %= <<= >>= >>>= &= ^= =	Từ phải qua trái

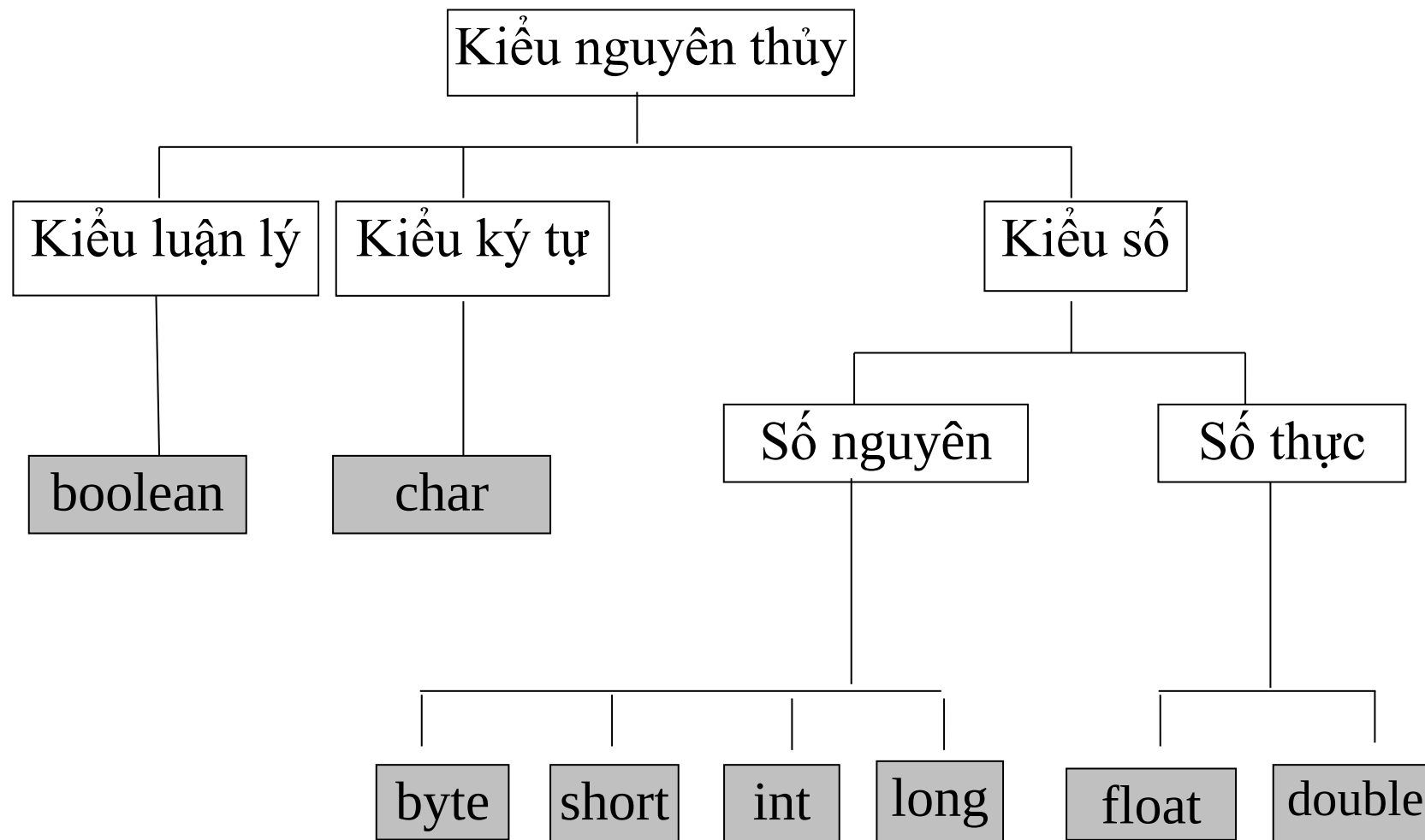


Kiểu dữ liệu nguyên thủy

- ❑ Kiểu dữ liệu:
 - ❑ Kiểu dữ liệu nguyên thủy (Primitive data type)
 - ❑ Kiểu dữ liệu tham chiếu hay dẫn xuất (reference data type)
- ❑ Kiểu dữ liệu cơ sở của Java bao gồm các nhóm sau: số nguyên, số thực, ký tự, kiểu luận lý (logic)
- ❑ Kiểu dữ liệu tham chiếu là các kiểu dữ liệu đối tượng. Ví dụ như: String, Byte, Character, Double, Boolean, Integer, Long, Short, Font,... và các lớp do người dùng định nghĩa.



Kiểu dữ liệu nguyên thủy





Kiểu dữ liệu nguyên thủy

□ Kiểu số nguyên

Kiểu	Kích thước	Khoảng giá trị
byte	8 bits	-256...255
short	16 bits	-32768...32767
int	32 bits	$-2^{32} \dots 2^{32} - 1$
long	64 bits	$-2^{64} \dots 2^{64} - 1$

□ Kiểu số thực

Kiểu	Kích thước	Khoảng giá trị
float	32 bits	-3.4e38...3.4e38
double	64 bits	-1.7e308...1.7e308



Kiểu dữ liệu nguyên thủy

- ❑ Kiểu **boolean**: Nhận giá trị true hoặc false
- ❑ Kiểu **char**: Kiểu ký tự theo chuẩn Unicode

Một số hằng ký tự:

Ký tự	Ý nghĩa
\b	Xóa lùi (BackSpace)
\t	Tab
\n	Xuống hàng
\r	Dấu enter
\”	Nháy kép
\’	Nháy đơn
\\	Số ngược
\f	Đẩy trang
\uxxxx	Ký tự unicode



Kiểu dữ liệu nguyên thủy

Một số lưu ý đối với các phép toán trên số nguyên:

- ❑ Nếu hai toán hạng kiểu *long* thì kết quả là kiểu *long*.
- ❑ Một trong hai toán hạng không phải kiểu *long* sẽ được chuyển thành kiểu *long* trước khi thực hiện phép toán.
- ❑ Nếu hai toán hạng đều không phải kiểu *long* thì phép tính sẽ thực hiện với kiểu *int*.
- ❑ Các toán hạng kiểu *byte* hay *short* sẽ được chuyển sang kiểu *int* trước khi thực hiện phép toán.



Kiểu dữ liệu nguyên thủy

Kiểu số thực (dấu chấm động):

- ❑ Kiểu *float* có kích thước 4 *byte* và giá trị mặc định là 0.0f
- ❑ Kiểu *double* có kích thước 8 *byte* và giá trị mặc định là 0.0d
- ❑ Khai báo và khởi tạo giá trị cho các biến kiểu dấu chấm động:

float x = 100.0/7;

double y = 1.56E6;



Kiểu dữ liệu nguyên thủy

Một số lưu ý với các phép toán trên số thực (dấu chấm động):

- ❑ Nếu mỗi toán hạng đều có kiểu dấu chấm động thì phép toán chuyển thành phép toán dấu chấm động.
- ❑ Nếu có một toán hạng là *double* thì các toán hạng còn lại sẽ được chuyển thành kiểu *double* trước khi thực hiện phép toán.
- ❑ Biến kiểu *float* và *double* có thể ép chuyển sang kiểu dữ liệu khác (trừ kiểu *boolean*).



Kiểu dữ liệu nguyên thủy

Kiểu ký tự:

- ☐ Biểu diễn các ký tự trong bộ mã Unicode
- ☐ 2¹⁶ = 65536 ký tự khác nhau : từ '\u0000' đến '\uFFFF'

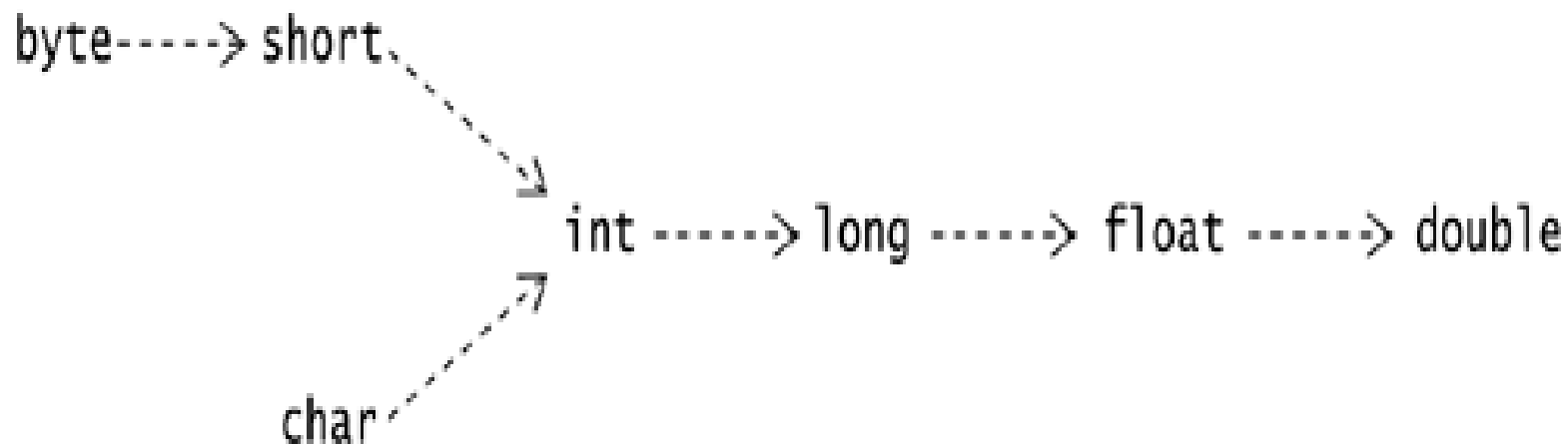
Kiểu luận lý (*boolean*):

- ☐ Hai giá trị: true hoặc false
- ☐ Giá trị mặc định: false
- ☐ Không thể chuyển thành kiểu nguyên và ngược lại



Kiểu dữ liệu nguyên thủy

- ❑ Khi gặp phải sự không tương thích kiểu dữ liệu chúng ta phải tiến hành chuyển đổi kiểu dữ liệu cho biến hoặc biểu thức



Ép kiểu rộng(widening conversion):



Kiểu dữ liệu nguyên thủy

- ❑ Toán tử ép kiểu: $\langle \text{tên biến} \rangle = (\text{kiểu_dữ_liệu}) \langle \text{tên_biến} \rangle;$
float fNum = 2.2;
int iCount = (int) fNum
- ❑ Ép kiểu rộng (widening conversion): Là quá trình làm tròn số từ kiểu dữ liệu có kích thước nhỏ hơn sang kiểu có kích thước lớn hơn.
- ❑ Ép kiểu hẹp (narrow conversion): Là quá trình làm tròn số từ kiểu dữ liệu có kích thước lớn hơn sang kiểu có kích thước nhỏ hơn



Cấu trúc dữ liệu mảng

❑ Kiểu mảng:

❑ Khai báo: **int**[] iarray; hoặc **int** iarray[];

❑ Cấp phát: iarray = **new int**[100];

❑ Khởi tạo:

❑ **int**[] iarray = {1, 2, 3, 5, 6};

❑ **char**[] carray = {'a', 'b', 'c'};

Chú ý: Luôn khởi tạo hoặc cấp phát mảng trước khi sử dụng

❑ Một số khai báo không hợp lệ:

int[5] iarray;

int iarray[5];



Cấu trúc dữ liệu mảng

- ❑ **Kiểu mảng:**

- ❑ Truy cập mảng

- ❑ `iarray[3] = 0;`

- ❑ `carray[1] = 'z';`

- ❑ Chú ý:

- ❑ Chỉ số của mảng được tính từ 0

- ❑ Lấy số phần tử mảng: `iarray.length`



Ví dụ

```
public class VariableDemo {  
    static int x, y;  
    public static void main(String[] args)  
    {  
        x = 10;  
        y = 20;  
        int z = x+y;  
        System.out.println("x = " + x);  
        System.out.println("y = " + y);  
        System.out.println("z = x + y =" + z);  
        System.out.println("Số nhỏ hơn là số:" +  
            Math.min(x, y));  
        char c = 80;  
        System.out.println("kí tự c là: " + c);  
    }  
}
```



Các câu lệnh cơ bản

- ❑ Nhập/xuất dữ liệu qua thiết bị chuẩn
- ❑ Các câu lệnh điều khiển dòng rẽ nhánh chương trình: lệnh chọn *if*, *if- else*, *switch*;
- ❑ Các câu lệnh lặp *while*, *do-while*, *for*, *for-each*
- ❑ Các lệnh thay đổi điều khiển chương trình: *break*, *continue*, *return*;



Nhập/xuất dữ liệu qua thiết bị chuẩn

- ❑ **Xuất dữ liệu:** *System.out*: cho phép in ra luồng ra thiết bị chuẩn. *System.out* có các phương thức:
 - ❑ *print()*: đầu ra của thông tin được truyền đi. Nó có thể lấy số, chuỗi và các đối tượng khác làm đối số;
 - ❑ *printf()*: đầu ra được định dạng. Định dạng văn bản được truyền bằng các chuỗi và đối số đặc biệt;
 - ❑ *println()*: đầu ra của thông tin được truyền đi và nguồn cấp dữ liệu đường truyền. Nó có thể lấy số, chuỗi và các đối tượng khác làm đối số;



Nhập/xuất dữ liệu qua thiết bị chuẩn

- ❑ **Nhập dữ liệu:** Để nhập dữ liệu từ thiết bị nhập chuẩn (như bàn phím,...), Java cung cấp lớp *Scanner*:

- ❑ Tạo đối tượng nhập (ví dụ **console**) và kết nối với thiết bị nhập chuẩn:

```
Scanner console = new Scanner(System.in);
```

- ❑ Để sử dụng lớp Scanner chúng ta cần import lớp Scanner đến dự án:

```
import java.util.Scanner;
```



Nhập/xuất dữ liệu qua thiết bị chuẩn

❑ Nhập dữ liệu: các phương thức của lớp *Scanner* :

Phương thức	Mô tả	Ví dụ
<i>nextInt()</i>	<i>Nhận số nguyên</i>	int a = console.nextInt();
<i>nextDouble()</i>	<i>Nhận số thực</i>	double b = console.nextDouble();
<i>next()</i>	<i>Nhận một chuỗi ký tự</i>	String str = console.next();
<i>nextLine()</i>	<i>Nhận một chuỗi ký tự trên dòng kế tiếp</i>	String str = console.nextLine();
<i>next().charAt(0)</i>	<i>Nhận một ký tự</i>	char ch = console.next().charAt(0);



Nhập/xuất dữ liệu qua thiết bị chuẩn

- ❑ **Ví dụ:** Chương trình tính tổng hai số nguyên được nhập từ bàn phím và hiển thị tổng đó ra màn hình:

```
import java.util.Scanner;

public class VariableDemo {
    public static void main(String[] args)
    {
        Scanner console = new Scanner(System.in);
        System.out.print("Enter a = ");
        int a = console.nextInt();
        System.out.print("Enter b = ");
        int b = console.nextInt();
        System.out.println();
        int c = a + b;
        System.out.printf("a + b = %d", c);
    }
}
```

Các câu lệnh điều khiển dòng rẽ nhánh chương trình

Câu lệnh if đơn giản

if (<Biểu thức điều kiện>)

<Câu lệnh>

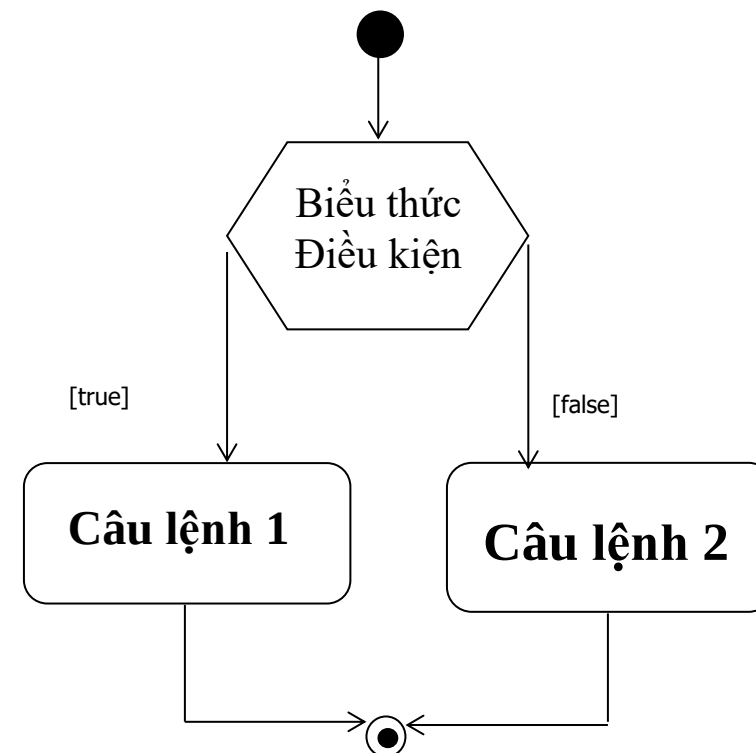
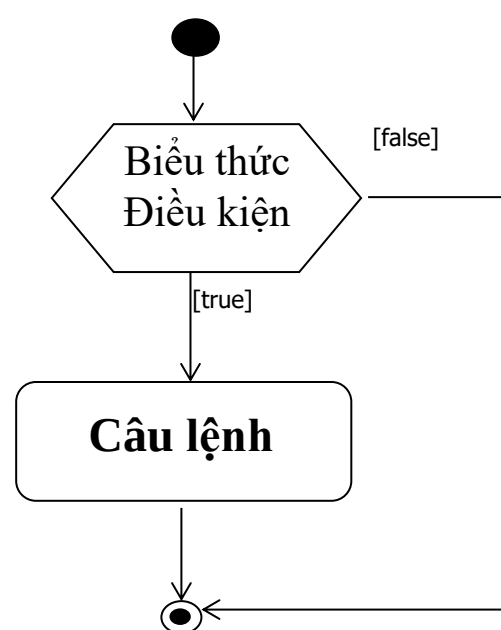
Câu lệnh if-else

if (<Biểu thức điều kiện>)

<Câu lệnh 1>

else

<Câu lệnh 2>



Trong đó: <Biểu thức điều kiện>: Biểu thức logic

<Câu lệnh>, <Câu lệnh 1>, <Câu lệnh 2>: Các câu lệnh mà MT cần thực hiện tương ứng với giá trị của B.Thức ĐK.



Các câu lệnh điều khiển dòng rẽ nhánh chương trình

Câu lệnh switch: Câu lệnh này cho phép rẽ nhánh theo nhiều nhánh tuyến chọn dựa trên các giá trị kiểu nguyên của biểu thức.

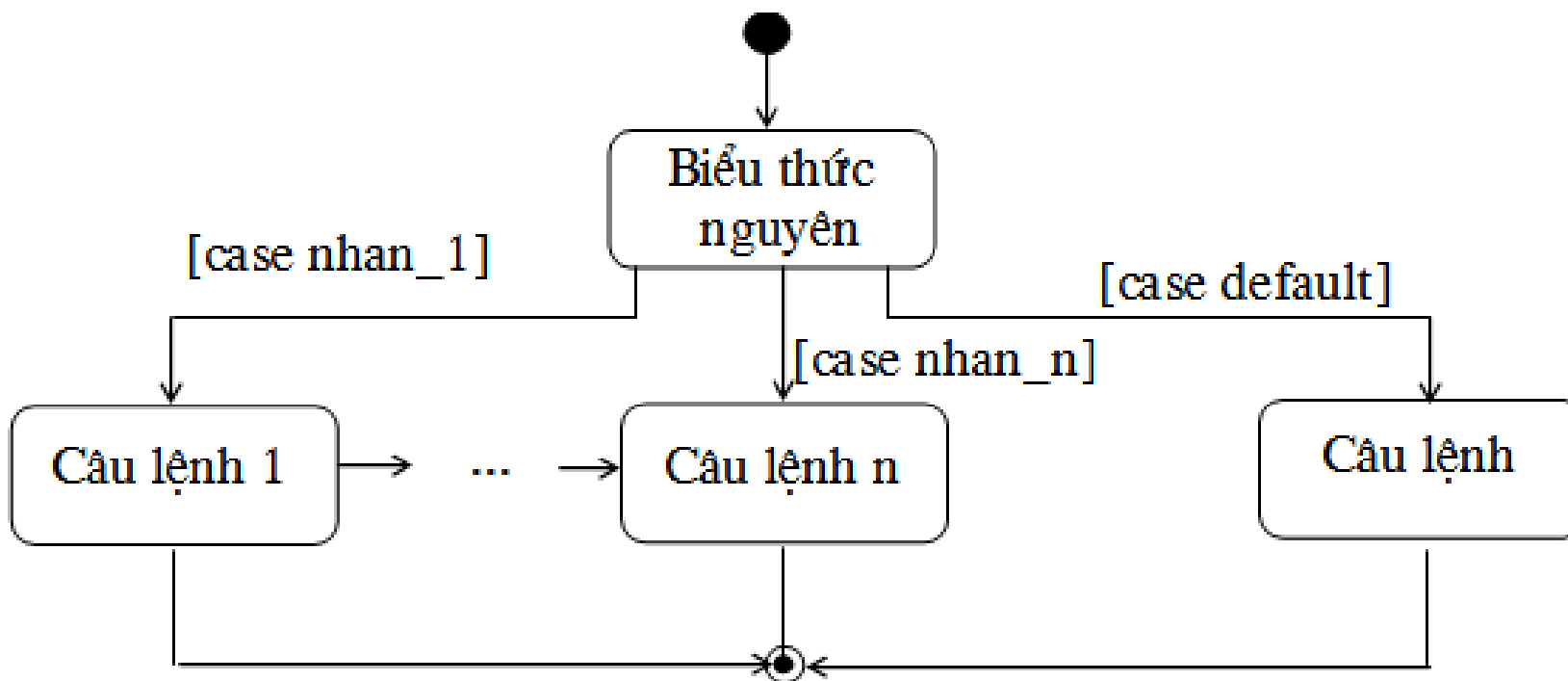
Cú pháp:

```
switch (<Biểu thức nguyên>) {  
    case nhan_1: <Câu lệnh 1>  
    case nhan_2: <Câu lệnh 2>  
    ...  
    case nhan_n: <Câu lệnh n>  
    [ default: <Câu lệnh > ]  
}
```




Các câu lệnh điều khiển dòng rẽ nhánh chương trình

Câu lệnh switch: Sơ đồ hoạt động.





Các câu lệnh lặp

Câu lệnh lặp (chu trình): cho phép một khối các câu lệnh thực hiện một số lần lặp lại.

Các cấu trúc điều khiển lặp trong Java: 4

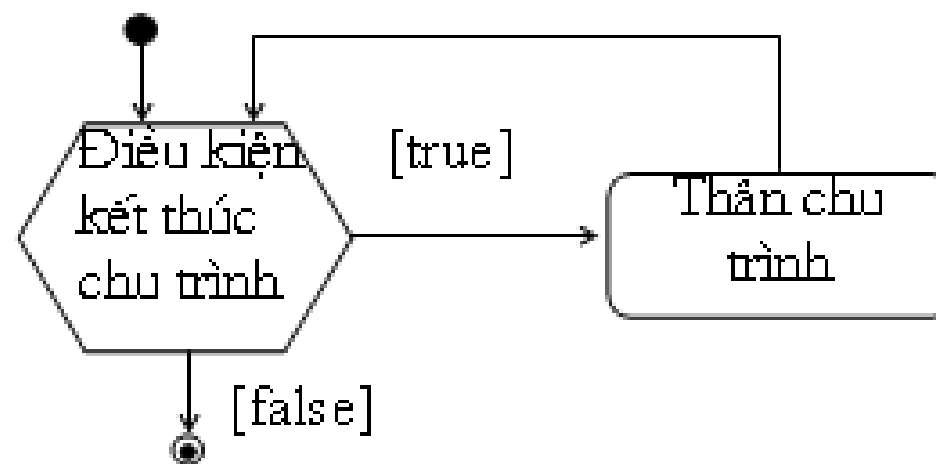
- Câu lệnh *while*
- Câu lệnh *do-while*
- Câu lệnh *for*
- Câu lệnh *for-each*



Câu lệnh while (chu trình while)

while (<Điều kiện kết thúc chu trình>)

<Thân chu trình>



❑ *Lưu ý:*

- ❑ Có thể <Thân chu trình> không được thực hiện lần nào, nếu ngay từ đầu <Điều kiện kết thúc chu trình> có giá trị *false*,
- ❑ <Điều kiện kết thúc chu trình> phải là biểu thức *boolean*.

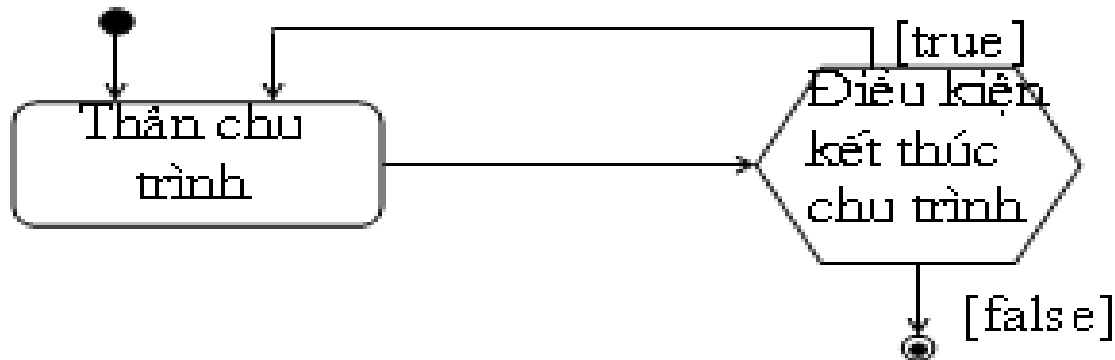


Câu lệnh do-while (chu trình do-while)

do

<Thân chu trình>

while (<Điều kiện kết thúc chu trình>)



Lưu ý: <Thân chu trình> phải thực hiện ít nhất một lần.



Câu lệnh for (chu trình for)

for (<B.Thức bắt đầu>; <Đ.Kiện lặp>; <B.Thức gia tăng>)

<Thân chu trình>

Trong đó:

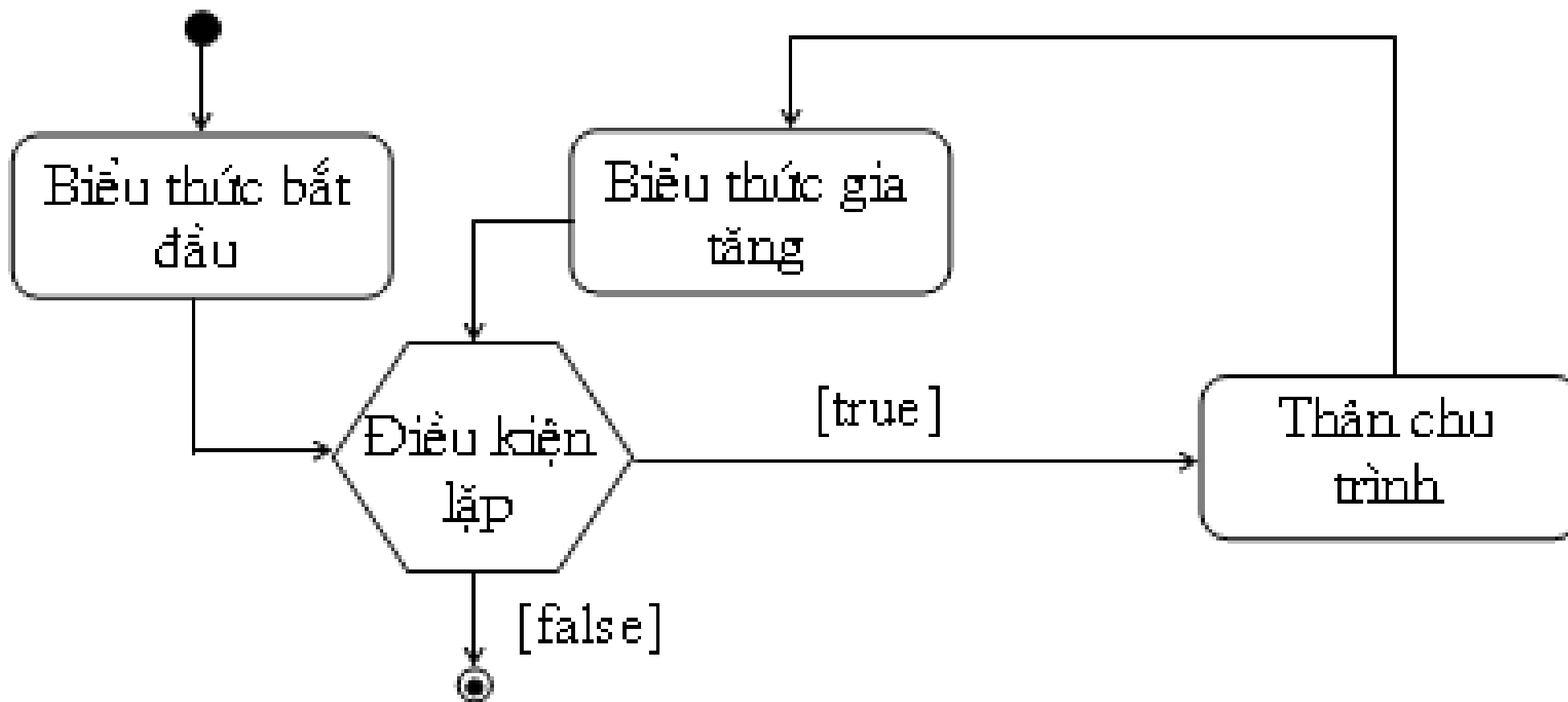
- ❑ <B.Thức bắt đầu>: khai báo và gán các giá trị khởi đầu cho các biến điều khiển quá trình lặp của chu trình.
- ❑ <Đ.Kiện lặp> là biểu thức boolean. Quyết định quá trình lặp sẽ được tiếp tục hay không.
- ❑ Sau mỗi lần thực hiện <Thân chu trình> thì <B.Thức gia tăng> lại được thực hiện.

Lưu ý: <Biểu thức bắt đầu> chỉ thực hiện đúng một lần ngay khi câu lệnh for bắt đầu thực hiện.



Câu lệnh for (chu trình for)

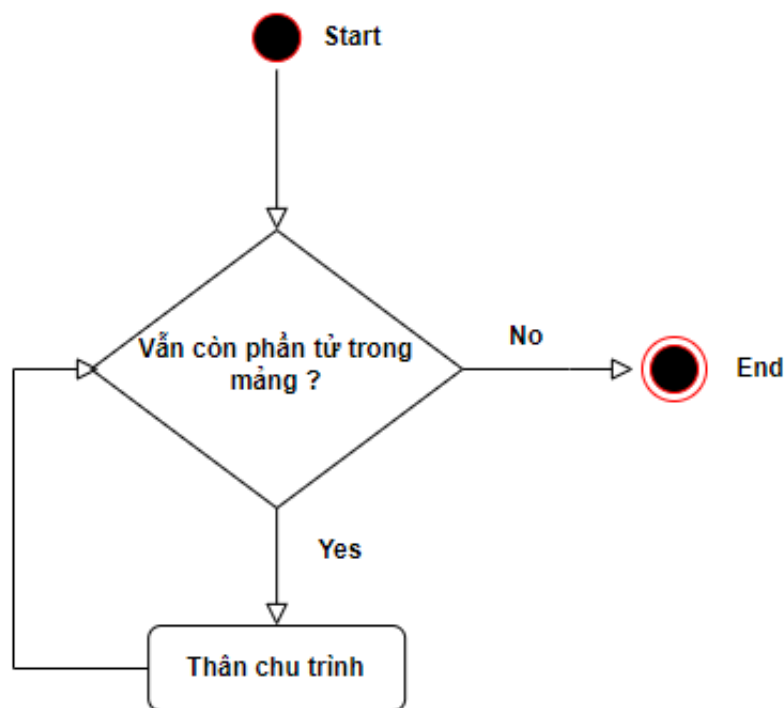
Sơ đồ hoạt động:





Câu lệnh for-each (chu trình for-each)

```
for (<kiểu dữ liệu> <tên biến chạy> : <tên mảng>) {  
    <Thân chu trình>  
}
```





Các câu lệnh chuyển vị

Java cung cấp bốn cách để chuyển điều khiển thực hiện của chương trình:

- ❑ *break*
- ❑ *continue*
- ❑ *return*
- ❑ *try-catch-finally* (*Xử lý ngoại lệ*)



BÀI TẬP TỔNG HỢP

Viết chương trình thực hiện các công việc sau:

- 1) Nhập vào một số nguyên dương n từ bàn phím ($2 \leq n \leq 100$)
- 2) Hiển thị ra màn hình danh sách tất cả các số nguyên tố $\leq n$
- 3) Hiển thị ra màn hình danh sách n số nguyên tố đầu tiên
- 4) Hiển thị ra màn hình n số Fibonacci đầu tiên (Số Fibonacci là số thoả mãn điều kiện: $F_0 = F_1 = 1$; $F_n = F_{n-2} + F_{n-1}$, $\forall n \geq 2$)



TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

Thank you!

THÁI NGUYÊN, THÁNG NĂM